🚀Proyecto Full Stack

Aplicación fullstack desarrollada con tecnologías modernas, separando claramente frontend y backend para una arquitectura escalable y mantenible.

📌 Descripción

Este proyecto consiste en una aplicación web completa que incluye:

🎨 Frontend moderno en React con TypeScript ⚙️ Backend robusto en Go 🗄️ Base de datos relacional con PostgreSQL 🔐 Autenticación basada en JWT

🖥️ Frontend

El frontend está desarrollado con un enfoque moderno, modular y tipado.

🛠️ Tecnologías usadas ⚛️ React + TypeScript 🌐 React Router → manejo de rutas SPA 📡 Axios → consumo de APIs 🧠 Zustand → manejo de estado global 🎨 Tailwind CSS → estilos rápidos y responsivos ✨ Características Arquitectura basada en componentes Manejo de estado global ligero y eficiente Navegación fluida sin recarga de página Consumo de API desacoplado UI responsive y moderna

⚙️ Backend

El backend está construido con Go, enfocado en rendimiento, simplicidad y escalabilidad.

🛠️ Tecnologías usadas 🐹 Go (Golang) → lenguaje principal 🗄️ PostgreSQL → base de datos relacional 🔐 JWT (JSON Web Tokens) → autenticación y autorización ✨ Características API RESTful Manejo de autenticación con tokens Conexión eficiente a base de datos Código organizado por capas (handlers, services, repositories, etc.) Alto rendimiento y bajo consumo de recursos

🔐 Autenticación

El sistema utiliza JWT para:

Login de usuarios Protección de rutas Validación de sesiones

Favor Revisar los README de la carpeta forntend y backend alli que encuentran las indicaciones de cada proyecto

🚀 Deploy Frontend con Nginx (Ubuntu)

1. Instalar dependencias

npm install

2. Generar build
npm run build

Esto generará una carpeta como:

dist/ (Vue, Vite, Angular)
build/ (React)

3. Copiar archivos a Nginx

Crear carpeta destino:
sudo mkdir -p /var/www/frontend

Copiar archivos (ejemplo con dist):
sudo cp -r dist/* /var/www/frontend/

Si usas React:
sudo cp -r build/* /var/www/frontend/

4. Configurar Nginx

Crear archivo:
sudo nano /etc/nginx/sites-available/frontend

Contenido:

```
server {
    listen 80;
    server_name tu-dominio.com;

    root /var/www/frontend;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

5. Activar configuración

```
sudo ln -s /etc/nginx/sites-available/frontend /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

6. Actualizar el frontend

Cada vez que hagas cambios:

```
npm run build
sudo rm -rf /var/www/frontend/*
sudo cp -r dist/* /var/www/frontend/
sudo systemctl reload nginx
```

✅ Resultado

Tu aplicación estará disponible en:
http://tu-dominio.com en este caso seria como localhost en fururas actualizaciones podriamos configurar usando un dominio + dns o deploy automático (CI/CD)

O Tambien podremos solo usar el comando npm run dev para correr la aplicacion del frontend

🚀 Backend API - Go + Fiber

Aplicación backend desarrollada con **Go (Golang)** utilizando el framework **Fiber**, con base de datos **PostgreSQL**, autenticación con **JWT** y despliegue mediante **Docker**.

📂 Estructura del Proyecto

```
backend/
├── db/                 # Configuración y conexión a base de datos
├── handler/            # Controladores (manejo de requests)
├── helpers/            # Funciones auxiliares
├── model/              # Modelos de datos
├── repositories/       # Acceso a datos
├── router/             # Definición de rutas
├── services/           # Lógica de negocio
├── resources/
│   └── pdf/            # Archivos PDF generados
├── main.go             # Punto de entrada
├── Dockerfile          # Imagen Docker del backend
├── docker-compose.yml  # Orquestación de servicios
├── ddl.sql             # Definición de esquema
├── dml.sql             # Datos iniciales
└── init.sh              # Script de inicialización
```

🔐 Variables de Entorno

Configura estas variables antes de ejecutar el proyecto:

```
# Frontend
URL_FRONTEND=http://localhost:5173

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=1020
DB_NAME=istla

# Servidor
SERVER_PORT=5002

# Seguridad
JWT_SECRET=theerudito

# Recursos
SOURCE_PATH=resources
```

```
IMAGE=imagen
PDF=pdf

# API URLs
API_URL_LOCAL=http://127.0.0.1:5002/api/v1/
```

🐳 Instalación de Docker

```
sudo apt update && sudo apt upgrade -y

sudo install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update

sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# Permisos
sudo usermod -aG docker $USER
newgrp docker

# Iniciar Docker
sudo service docker start
```

🌐 Instalación de Nginx

Servidor web para producción:

```
sudo apt install nginx -y
```

🔥 Configuración de Firewall (UFW)

```
sudo apt install ufw -y

sudo ufw enable

# Permitir puertos necesarios
sudo ufw allow 80
sudo ufw allow 443
sudo ufw allow 8080
```

```
sudo ufw allow 22

# Regla completa para Nginx
sudo ufw allow "Nginx Full"

sudo ufw status numbered
```

🚀 Uso con Docker
▶️ Crear y levantar contenedores

```
docker-compose up --build
```

🛑 Detener y eliminar contenedores

```
docker-compose down -v
```

🧰 Portainer (Opcional)

Interfaz web para administrar Docker.

▶️ Instalación

```
docker run -d \
-p 8000:8000 \
-p 9443:9443 \
--name portainer \
--restart=always \
-v /var/run/docker.sock:/var/run/docker.sock \
-v portainer_data:/data \
portainer/portainer-ce:lts
```

🌐 Acceso
https://localhost:9443
Usuario: admin
Configura la contraseña en el primer acceso

🚀 Estado del Servidor (Fiber)

Cuando la API está corriendo, verás algo como:

```
┌──────────────────────────────────────────────────────────┐
│                  🚀 Fiber v2                              │
│                                                           │
│   Server running on http://localhost:5002                │
│   Environment: development                                │
│   PID: 12345                                              │
└──────────────────────────────────────────────────────────┘
```

🌐 Base URL
http://localhost:5002/api/v1/

📌 Notas
Verifica que los puertos estén disponibles antes de ejecutar Docker.
Configura correctamente las variables de entorno.
Ajusta las URLs según tu entorno (local o red).

🔐 Autenticación

Método	Endpoint	Descripción	Auth
POST	/login	Iniciar sesión	❌
POST	/register	Registrar usuario	❌

👤 Profiles

Método	Endpoint	Descripción	Auth
GET	/profiles	Obtener perfiles	✅ JWT

📝 Posts

Método	Endpoint	Descripción	Auth
GET	/post	Obtener todos los posts	✅ JWT
GET	/post/get_by_user/:id	Obtener posts por usuario	✅ JWT
POST	/post	Crear nuevo post	✅ JWT
PUT	/post	Actualizar post	✅ JWT
DELETE	/post/:id	Eliminar post	✅ JWT

📁 Recursos

Método	Endpoint	Descripción	Auth
GET	/resources/:folder/:file	Obtener archivos (img/pdf)	❌

🔐 Autenticación (JWT)

Los endpoints protegidos requieren un token JWT en el header:

Authorization: Bearer <tu_token>

Modelo Login
```
{
   "identificacion" : "1721457495",
   "password" : "123456"
}
```

Modelo Register

{
"identificacion" : "1721457495",
"nombres" : "JORGE",
"apellidos" : "LOOR",
"email" : "erudito.tv@gmail.com",
"password" : "123456",
"id_perfil" : 1
}

Modelos POST
Tipo Contendio FormData
description : "HOLA",
usuario_id : "1"
file : el archivo pdf
usuario_creacion : "JORGE"

Modelos PUT
Tipo Contendio FormData
post_user_id" : "1",
description : "HOLA",
usuario_id : "1"
file : el archivo pdf en editar puede envia o no la imagen
usuario_modificacion : "JORGE"